She was ahead of her time.

Language == (Code && 💕)

By R. E. Warner

> *"Our family are an alternate stratification of poetry and mathematics."*
> *—Ada Lovelace, in a letter to Andrew Crosse*

Not Political, *Poetical* Science

I wrote my 33 character Twitter bio many years ago—way back when Twitter was fun, kids! "*Writer of story, poetry and code.*" I wrote that line before I thought too hard about what it meant. I'm thinking hard about it now because something about it has changed. The line between code and language is blurring (again).

```
For those non-programmer types out there, that `==` in the
title is a special sign in coding that means to check that two
variables are equal—you can't use `=` because that's what
*sets* the variable in the first place.
```

The first programmer was a writer. Ada Lovelace wrote the first algorithm for Babbage's Analytical Engine in 1843—in prose, essentially. Annotations to a translation of an Italian paper, extended into something that described what a machine *could* do if someone told it what to mean. She was writing a specification. She was writing natural language instructions for a machine that didn't fully exist yet. She never touched a compiler. There were no compilers. There was only language, precise and visionary, reaching toward a machine that would take another

century to arrive. Lovelace was Lord Byron's daughter, raised deliberately *away* from poetry and toward mathematics—and she still couldn't separate them. She called it poetical science. The first programmer refused to accept that poetry and mathematics were different languages. She was right.

Then writing for machines got mechanical—holes punched in heavy stock paper punch cards fed into machines that only cared about sequence and nothing else. You fed them into computers to make the computer know something. They were the cartridge, the disk, the CD, of the day. My mother worked as a technical writer in that era, and she relates with chagrin watching someone drop a box of punch cards. The sequence, the *program*—gone, scattered across a linoleum floor or sidewalk. Someone had to get on their hands and knees and reconstruct the program, the order of the universe, card by card. That's not programming. That's liturgy. The machine only cared about the sequence; didn't care about your intentions. The languages of that era—assembly, FORTRAN, COBOL—required the programmer to think like the machine. Registers, opcodes, memory addresses. A small priesthood, initiated through suffering. How do you error correct?

Much, much worse than a `git push` gone wrong.

Compilers arrived thanks to Grace Hopper🅆 and the conversation got a little more civilized with "high-level" programming; closer to English. She built a layer of abstraction between human intention and machine execution, and the priesthood exhaled. I remember coming home from college, stoked that I had learned how to manipulate Unix (The operating system at the heart of Macs, iPhones and most of the web). My father, ever whimsical, said he worked down the hall from the guy who created Unix. He wasn't joking. He worked down the hall from Dennis Ritchie🅆 when he was at Bell Labs. I couldn't believe it—someone invented Unix? (Let us not forget Ken Thompson🅆.) Then C. Then C++. Then Java. The dream, barely

concealed in each iteration, was always the same: what if we could just say what we meant for the machine to do?

COBOL made a serious attempt in 1959. Business logic in plain English sentences, readable by non-programmers—in theory. That didn't quite work out. But the dream persisted. Ruby arrived and you could write code that read like English—or at least like English with very strict grammar and no patience for ambiguity. Python read like pseudocode made real. JavaScript spread everywhere, flawed and beloved, the Latin of the web.

Here's the thing I never found strange about any of this: I only ever thought of language as language. To make it more or less grammatical depending on the subject was always obvious—even when I was speaking, I was thinking about my audience. Who is listening? Who is reading? The grammar adjusts accordingly. (That's a skill)

Writing code requires very strict grammatical rules—computers are just bad at inference—we're not, we fill in blanks. I write English with considerably looser than my code. As noted by linguist, professor emeritus at MIT, Noam Chomsky, "Colorless green ideas sleep furiously" is a grammatically correct sentence that literally doesn't possess semantic meaning. But we inferential machines imbibe it with meaning—you felt it before you analyzed it. Code doesn't allow for this. A compiler rejects it without ceremony. Language holds it, turns it over, finds the light inside it. Poetry lives in exactly that gap between grammatical and meaningful, between what the rules permit and what the mind receives.

The entire history of programming languages is the history of closing that gap—not by loosening code's grammar, but by making the machine climb toward ours. That summit is arising.

From Spec to Software

Natural language interfaces for coding agents aren't the end of that climb. But still, you describe what you want in plain language. The agent writes the code, runs the tests, catches the errors, refactors the output. And the tools are starting to formalize this. Speckit 🐙, GitHub's specification kit, treats the English-language spec as the primary artifact—the thing from which software is derived. Not a document stapled to the back of a pull request. The spec *is* the software, upstream of every line of code that follows.

This has been coming for a long time. Test-driven development—TDD🆆, the practice of writing tests before writing code—was already pointing this direction. You describe the desired behavior first, precisely, in terms the machine can verify. Then the code follows. The description precedes the implementation. The intention leads, and execution catches up.

Git, the version control system every developer lives inside, is a writing discipline in disguise. A good commit message is a sentence that says exactly what changed and why. A pull request is an argument—here is what I did, here is why it is correct, here is what you should check. Code review is editing. The whole apparatus is language, structured and purposeful, carrying intention from one mind to another across time.

The developer curriculum of the future is a writing curriculum. How to write a product requirements document. How to write a specification. Maybe how to write pseudocode? Take a course in symbolic logic. Definitely know math (strictest grammar there is!), design patterns, design systems. How to write a test before you write the code. How to write a commit message that your collaborator—human or machine—can act on. How to write a prompt that gets you what you actually meant.

These are not soft skills orbiting the technical core. They *are* the technical core.

The Poetical Skill

Writing for machines means being specific. It doesn't mean typing faster or producing more. It means developing the capacity to judge what's in front of you.

Because here's what working with a coding agent actually feels like: you are not making the software. You are art directing it. You describe the shot. You review the take. You say *no, that's not it*—and you redirect. A great art director can walk onto a set and know immediately when the light is wrong without being able to operate the equipment. But they got there by looking at thousands of photographs. By developing an eye. You cannot shortcut that formation.

The same is true here. A developer working with an agent needs taste—the capacity to recognize when the output is technically correct but wrong or simply not elegant. When it works but it's brittle. When it passes the tests but it's going to be a nightmare to maintain. When it solves the stated problem but missed the actual one.

And taste is downstream of skepticism. Skepticism is the root. You have to walk up to the output and say *I don't trust this yet*. You have to resist the pull of the plausible. Jack Clark of Anthropic says that they have seen their developers turn more and more work over to agents. Their developers interrupt the agents to approve steps less and less. LLM output is very good at being plausible—that's structurally what it is: the most statistically reasonable next token, the smoothest path through the probability space. It won't likely surprise you with a solution the way a real solution surprises you, because surprise requires a willingness to be wrong, to go somewhere uncomfortable, to follow a problem past the point where

it's safe. LLMs aren't really known for that.

The output defaults to anodyne. Smooth, competent, inoffensive, forgettable. It sands off the edges. A developer with taste walks in and says *no*—and knows why. Skepticism is teachable. Not through being fooled, but through practice—close reading, Socratic questioning, finding patterns, the discipline of asking *what is this actually doing* before accepting what it appears to be doing. Every good writing teacher and every good code reviewer has always been doing this. It just got more urgent.

In the 6th grade, my inventive English teacher Mrs. Ryan, set out to teach us expository writing—just the kind needed for dumdum machines. She asked us all to write instructions to make a peanut butter and jelly sandwich. After we completed the assignment she stood behind the counter and followed students' instruction *literally*. If you didn't tell her to open the jar, she would bang the knife on the lid. If you didn't tell her how much spread to use, she'd use a little or too much. Anyway, a brilliant demonstration which stuck with me for years. It's the exact kind of skill new developers need.

Before there were compilers, before there were punch cards, before there were programming languages at all, there were human computers—people who took natural language instructions from scientists and mathematicians and produced numerical output by hand. That was the interface. That was the original API. The machine eventually replaced them, and for eighty years we taught ourselves to speak machine. Now the machine is learning to speak human again. We have come full circle—back to Ada Lovelace's prose, back to natural language as the primary interface, back to the beginning.

My bio says *writer of story, poetry and code*. I used to think that was a description of range—look how many modes I can work in. I understand now that it was never three things. It was always one thing, with the semantics and grammar

dialed to different levels depending on who was listening. Story for humans with time. Poetry for humans with feeling. Code for machines with no patience for ambiguity whatsoever. But, the machine is developing patience. The grammar is loosening at that end of the dial.

Which means the person who knows how to write—precisely, structurally, ironically?—with intention and taste and earned skepticism, is now the person who can drive the machine at every level. Specification. Prompt. Test. Commit. Story. The skill that was always called soft is now the technical skill. The thing that was always central is now, finally, legible as such—well, semi-legible. Excuse me while I go use a pencil to scribble some pseudocode to scan into an LLM for it to turn into a program.